Proposed resolution for CWG2692 Static and explicit object member functions with the same parameter-type-lists

Document #:	P2797R0
Date:	2023-02-10
Project:	Programming Language C++
Audience:	Core

Reply-to:	Gašper Ažman <gasper.azman@gmail.com> Barry Revzin <barry.revzin@gmail.com>

Contents

- 1 Abstract
- 2 Discussion
  - 2.1 The actual problem, explained
- 3 Resolution
- 4 Caveats if we chose option 1
- 5 Other related issues
- 6 Does this break code?
- 7 Drive-by wording changes
- 8 Wording
- 9 Other considerations
  - 9.1 Getting the pointers to these functions
  - 9.2 Mangling
- 10 Acknowledgements
- 11 References

§ 1 Abstract

This document proposes a resolution to [CWG2692]. This also touches the proposed resolution to [CWG2687].

§ 2 Discussion

Consider the example

```
struct A {
  static void f(A); // #A
  void f(this A);   // #B

  static void e(A const&); // #C
  void e() const&;         // #D

  void g() {
    // static + explicit memfn
    (&A::f)(A()); // #1
    f(A());       // #2
    (&A::f)();    // #3

    // static + implicit memfn
    (&A::e)(A()); // #4
    e(A());       // #5
    (&A::e)();    // #6
  }
};

void h() {
  // static + explicit memfn
  (&A::f)(A()); // #7
  f(A());       // ill-formed
  (&A::f)();    // #8

  // static + implicit memfn
  (&A::e)(A()); // #9
  e(A());       // ill-formed
  (&A::e)();    // #10
}
```

§ 2.1 The actual problem, explained

The options:

— Option 1: *#1* and *#2*, *#4* and *#5* behave the same, *#3* and *#6* are ill-formed.

— Option 2: *#1* is ambiguous, *#2* calls the static, *#4* is ambiguous, *#5* calls the static.

Explanations of the two options:

Option 1: the synthesized argument list for the *unqualified function call* `(&A::f)` is actually `((A&)*this, A{})` (per 12.2.2.2.2 [over.call.func]/3), therefore only *#A* is a viable candidate for *#1* and *#2*.

Option 2: If `(&A::e)` instead of resolving on the original overload set first takes the address of *all* of the overload set, and then resolves the call expression on those pointers, then *#1* and *#4* are ambiguous, and do something different than *#2* and *#3*.

§ 3 Resolution

EWG chose option 2. Call-of-address-of-overload-set expressions like `(&function-id-expression)(expr-list)` decay the overload set to pointers to the elements of the overload set before choosing best-match.

12.2.2.2.1 [over.match.call.general]/2 just refers to 12.2.2.2.2 [over.call.func]/3.

§ 4 Caveats if we chose option 1

The proposed resolution (of affirming the status quo) means that there is a difference in this case:

```
auto identity(auto x) { return x; }

struct C {
  void f(this C); // note: not an overload set

  auto g() {
            (&C::f)(C{}); // always ill-formed
    identity(&C::f)(C{}); // OK
  }
};
```

§ 5 Other related issues

The core issue proposes to diagnose #A and #B as conflicting. We propose not doing that. It would create an inconsistency between explicit and implicit member functions, as well as remove a useful feature, since a call expression for either is never ambiguous. EWG affirmed this too.

```
void l() {
  A::f(A{}); // calls #A
  A{}.f();   // calls #B
}
```

§ 6 Does this break code?

No, there is already implementation divergence with implicit member functions.

GCC, clang and icx reject *#4* and *#6* with:

```
"reference to overloaded function could not be resolved, did you mean to
        call it?"
```

MSVC accepts *#4* and rejects *#6*:

```
MSVC error C2352: 'B::f': a call of a non-static member function requires
        an object.
```

ICC accepts both *#4* and *#6*. Our reading of 12.2.2.2.2 [over.call.func]/3 is that MSVC is correct.

§ 7 Drive-by wording changes

Due to the fact that approximately nobody is aware that *unqualified function call* and *qualified function call* are terms which have nothing to do with *unqualified-ids* or *qualified-ids*, we may want to fix the term of "(un)qualified function call" in the standard (editorially) and replace it with *objectish function call* for `x.f()` and `px->f()`, and *nonobjectish function call* for `N::f()`, due to the general confusion of "qualified" referring to `pobj->` or `obj.` instead of namespace qualification.

The authors would like to thank Davis for his amazing terminological suggestion.

These terms are only used 1 and 2 times, respectively, in 12.2.2.2.2 [over.call.func], so they don't have to be short.

The reasoning for the new term is that `->` and `.` are operators for *class member access*.

If the fact that `N::f(x)` is an *unqualified function call* surprises you, then you agree.

§ 8 Wording

Thanks to Davis Herring, the plan is:

1. Do not merely strip the `&` in 12.2.2.2.1 [over.match.call.general]/2; remember that it was there.
2. Do not add an implicit object parameter in 12.2.2.1 [over.match.funcs.general]/2 for static member functions when called with `&`.
3. Do not add an implied object argument in 12.2.2.2.2 [over.call.func]/3 if the `&` was there.
4. Back in 12.2.2.2.1 [over.match.call.general]/2, adjust the rejection mechanism.
5. Ensure that 7.6.1.3 [expr.call]/7 does the right thing (it does).

In 7.5.4.1 [expr.prim.id.general]/3:

An *id-expression* that denotes a non-static data member or ~~non-static~~implicit object member function of a class can only be used:

(3.1) — as part of a class member access in which the object expression refers to the member's class or a class derived from that class, or

(3.2) — to form a pointer to member (7.6.2.2 [expr.unary.op]), or

(3.3) — if that *id-expression* denotes a non-static data member and it appears in an unevaluated operand.

[Example 3:

```
struct S {
  int m;
};
int i = sizeof(S::m);           // OK
int j = sizeof(S::m + 42);      // OK
```

— end example]

Strike 7.6.1.3 [expr.call]/2 (every part of it is redundant with some other wording):

For a call to a non-static member function, the postfix expression shall be an implicit (11.4.3 [class.mfct.non.static], 11.4.9 [class.static]) or explicit class member access (7.6.1.5 [expr.ref]) whose *id-expression* is a function member name, or a pointer-to-member expression (7.6.4 [expr.mptr.oper]) selecting a function member; the call is as a member of the class object referred to by the object expression. In the case of an implicit class member access, the implied object is the one pointed to by this.

In 11.4.3 [class.mfct.non.static]/2:

When an *id-expression* (7.5.4 [expr.prim.id]) that is ~~not~~neither part of a class member access syntax (7.6.1.5 [expr.ref]) ~~and not used to form a pointer to member~~ nor the unparenthesized operand of the unary `&` operator (7.6.2.2 [expr.unary.op]) is used where the current class is `X` (7.5.2 [expr.prim.this]), if name lookup (6.5 [basic.lookup]) resolves the name in the *id-expression* to a non-static non-type member of some class `C`, and if either the *id-expression* is potentially evaluated or `C` is `X` or a base class of `X`, the *id-expression* is transformed into a class member access expression (7.6.1.5 [expr.ref]) using `(*this)` as the postfix-expression to the left of the `.` operator.

In 12.2.2.1 [over.match.funcs.general]/2:

The set of candidate functions can contain both member and non-member functions to be resolved against the same argument list.

So that argument and parameter lists are comparable within this heterogeneous set, a member function that does not have an explicit object parameter is considered to have an extra first parameter, called the implicit object parameter, which represents the object for which the member function has been called. For the purposes of overload resolution, both static and non-static member functions have an object parameter, but constructors do not.

If a member function is

— an implicit object member function that is not a constructor, or

— a static member function and the argument list includes an implied object argument,

it is considered to have an extra first parameter, called the implicit object parameter, which represents the object for which the member function has been called.

In 12.2.2.2.1 [over.match.call.general]/2:

If the *postfix-expression* is the address of an overload set, overload resolution is applied using that set as described above. [Note: No implied object argument is added in this case. – end note] If the function selected by overload resolution is ~~a non-static~~an implicit object member function, the program is ill-formed. [Note 1: The resolution of the address of an overload set in other contexts is described in 12.3 [over.over]. —- end note]

In 12.2.2.2.2 [over.call.func]/3:

In unqualified function calls, the function is named by a *primary-expression*. The function declarations found by name lookup (6.5 [basic.lookup]) constitute the set of candidate functions. Because of the rules for name lookup, the set of candidate functions consists ~~(1)~~either entirely of non-member functions or ~~(2)~~ entirely of member functions of some class `T`. In ~~case (1)~~the former case or if the *primary-expression* is the address of an overload set, the argument list is the same as the *expression-list* in the call. ~~In case (2)~~Otherwise, the argument list is the *expression-list* in the call augmented by the addition of an implied object argument as in a qualified function call. If the current class is, or is derived from, `T`, and the keyword `this` (7.5.2 [expr.prim.this]) refers to it, then the implied object argument is `(*this)`. Otherwise, a contrived object of type `T` becomes the implied object argument; if overload resolution selects a non-static member function, the call is ill-formed.

[Example 1:

```
  struct C {
    void a();
    void b() {
      a();                // OK, (*this).a()
    }

+   void c(this const C&);   // #1
+   void c()&;               // #2
+   static void c(int = 0);  // #3

+   void d() {
+     c();               // error: ambiguous between #2 and #3
+     (C::c)();          // error: as above
+      (&(C::c))();        // error: cannot resolve address of overloaded
        this->C::c [over.over]
+     (&C::c)(C{});      // selects #1
+        (&C::c)(*this);       // error: selects #2, and is ill-formed
        [over.match.call.general]/2
+     (&C::c)();         // selects #3
+   }

    void f(this const C&);
    void g() const {
```

```
    f();                // OK, (*this).f()
        f(*this);                   // error: no viable candidate for
      (*this).f(*this)
    this->f();          // OK
  }

  static void h() {
      f();                      // error: contrived object argument, but
       overload resolution
                        // picked a non-static member function
    f(C{});             // error: no viable candidate
    C{}.f();            // OK
  }

  void k(this int);
  operator int() const;
  void m(this const C& c) {
    c.k();              // OK
  }
};
```

— end example]

§ 9 Other considerations

§ 9.1 Getting the pointers to these functions

While `static_cast<void(*)(B const&)>(&B::f)` will work to deliver the static version, the same for `A` will be ambiguous. This is ok, we already have situations in the language where `static_cast` is less powerful in overload resolution than a call expression. The solution is to use a lambda.

§ 9.2 Mangling

The resolution means that functions with an explicit object parameter must be mangled differently than a static member function with the same formal parameter list, which realistically should be the case anyway.

§ 10 Acknowledgements

The authors would like to thank Davis Herring, for diligently working through all the issues and examples.

§ 11 References

[CWG2687] Matthew House. 2023-01-16. Calling an explicit object member function via an address-of-overload-set.
https://wg21.link/cwg2687

[CWG2692] Matthew House. 2023-01-16. Static and explicit object member functions with the same parameter-type-lists.
https://wg21.link/cwg2692